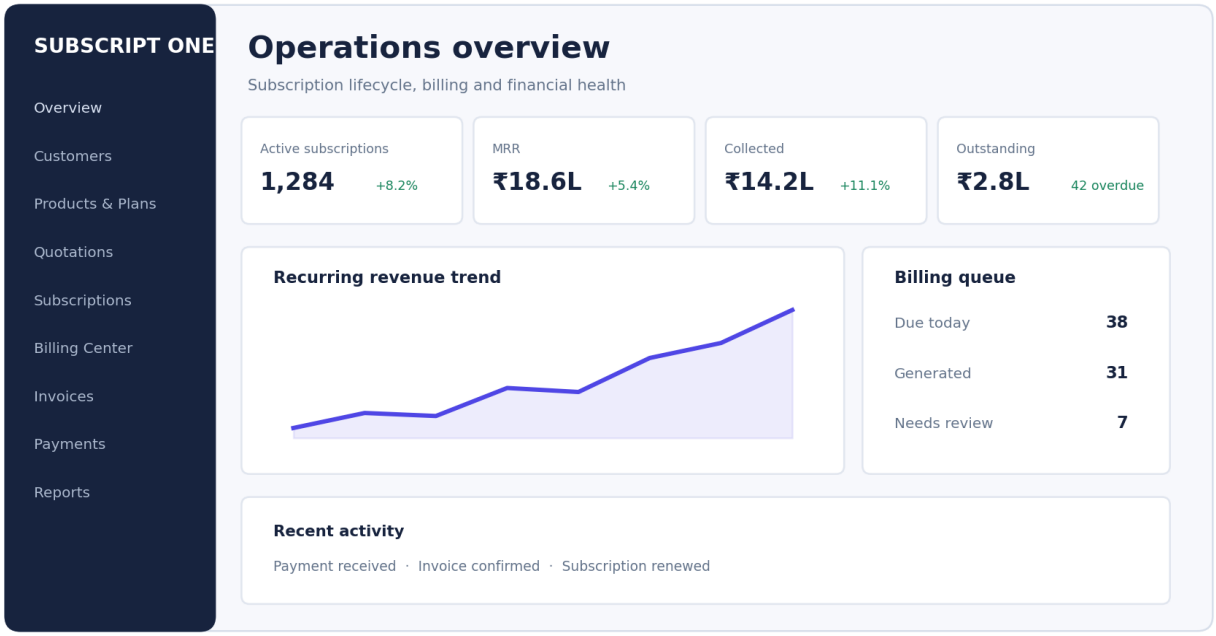
SUBSCRIPT ONE

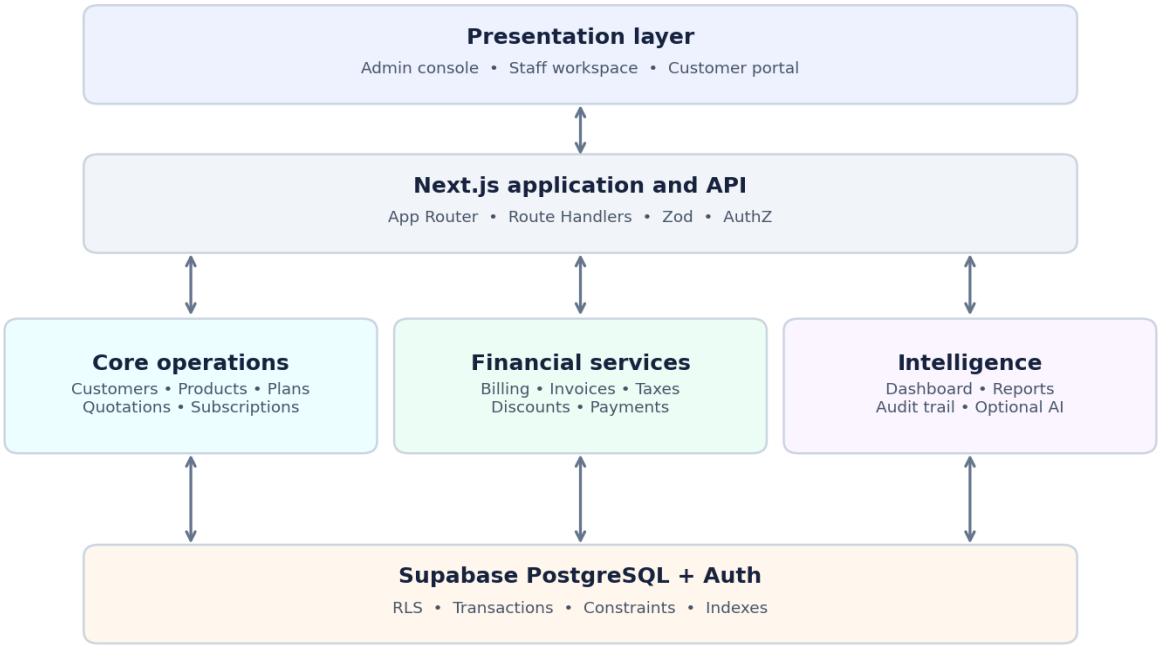
Combined Subscription Management
& Operations Command Center

Six-hour hackathon blueprint • Four-member team • Shared GitHub repository



Design validated against the supplied Brightloop Software problem statement. This is a blueprint, not a claim that the application has already been built or tested.

Architecture and lifecycle diagrams



One connected subscription lifecycle



Transactions + unique billing-period keys prevent duplicate invoices; audit events preserve traceability.

> **Decision:** Merge Idea 1 (complete subscription management) and Idea 5 (subscription command center) into one coherent product: **Subscript One**. The complete business system is the foundation; the command center is the operational experience over the same records.



Executive decision

Subscript One — Subscription Management & Operations Command Center manages products, variants, plans, customers, quotations, subscriptions, recurring billing, invoices, taxes, discounts, payments and finance reporting. Its command-center layer makes the same records actionable through a billing queue, renewal visibility, exception handling, activity history and financial KPIs.

Core promise: Operate the complete subscription business in one place; automate repetitive billing work; and give staff and finance an accurate, current view of the business.

Feasibility qualification: Six hours is sufficient for a carefully scoped, working MVP and demonstrable vertical slice. It is not sufficient to guarantee production-grade implementation of every edge case, email delivery, background scheduling, high-volume performance and every operational control. This blueprint distinguishes required behavior from the six-hour implementation strategy and identifies what must be tested rather than claimed.

1. Source requirements and interpretation

The primary source is the supplied *Subscription Management System* problem statement for Brightloop Software. It describes a unified platform for the full subscription lifecycle, with admin, internal staff and customer access. The accompanying team prompt sets hard constraints: six hours, exactly four members, one shared GitHub repository, simultaneous work, AI coding agents permitted, and analysis before implementation.

The problem statement explicitly requires:

• Customer registration; unique email; password longer than eight characters containing uppercase, lowercase and special character; password reset link to registered email.

• Admin, staff and customer access; only admin can add staff; customers see only their own records.

• Products with name, type, sale price, cost price and notes; variants with price additions.

• Plans with daily, weekly, monthly or yearly billing, minimum quantity, availability dates, and plan-level closure/pause/renewal behavior.

• Quotation templates with plan/product lines and validity period.

• Subscription number, customer, plan, dates, payment terms and lines with product, quantity, unit price, taxes and amount; lifecycle Draft → Quotation → Confirmed → Active → Closed.

• Invoices generated from subscriptions with customer details, lines, taxes and totals; Draft → Confirmed → Paid; confirm, cancel, send and print; no duplicate or skipped billing.

• Admin-managed fixed/percentage discounts with minimum purchase or quantity, validity dates, usage limits, and automatic application to eligible next invoices.

• Payments recorded with method, amount, date and note against the correct invoice.

• Visibility of paid and outstanding invoices, overdue follow-up, and finance reporting for active subscriptions, revenue, payments and overdue invoices.

• Pages responding in under two seconds with thousands of subscriptions.

Interpretation rules

• Preserve the required lifecycle and capabilities; do not replace them with an AI-only or analytics-only product.

• Treat the finance lead as staff with finance/reporting permissions unless organizers specify a distinct role. The problem statement describes the finance lead's needs but does not mandate a separate authentication role.

• For the six-hour demo, billing automation is a real server-side batch operation initiated by an authorized staff user. Scheduled production execution is a later deployment capability unless a scheduler is configured and tested.

• Invoice sending and password reset email are real email-delivery requirements. Configure and test a verified email provider. If unavailable during the hackathon, disclose the limitation and use a clearly labeled demo fallback rather than claiming delivery works.

• The source does not prescribe tax jurisdiction, tax-inclusive/exclusive behavior, currency conversion, payment gateway or refund policy. These must be configured or clarified; do not invent legal tax behavior.

2. Merged product concept

What Idea 1 contributes

The complete subscription management backbone: customer and staff management, products, variants, plans, quotation templates, subscriptions, invoices, discounts, payments, taxes and reports.

What Idea 5 contributes

The command-center experience: operational overview, billing queue, renewals and exceptions, recent activity, actionable financial KPIs, and cohesive navigation.

Product pillars

• **Manage:** maintain customers, products, variants, plans, quotation templates, subscriptions and access permissions.

• **Operate:** move quotations through approval and conversion; activate, pause, renew and close subscriptions; create, confirm, cancel, send and print invoices; record payments.

• **Automate:** calculate billing periods and amounts deterministically; process due subscriptions; apply eligible discounts; prevent duplicate invoices; update balances and statuses.

• **Understand:** show active subscriptions, collected revenue, outstanding balances, overdue invoices, renewals and activity.

Differentiation

The differentiator is the connection between a business action and its financial consequence. A recorded payment updates the payment ledger, invoice balance/status, outstanding totals, dashboard metrics and audit history. The command center surfaces the next operational action rather than merely storing records.

AI policy

AI is optional and non-critical. If added after core requirements pass, it may summarize server-computed metrics or explain overdue/renewal priorities using read-only, permission-filtered data. An LLM must never calculate tax, discount eligibility, invoice totals, billing dates or payment balances.

3. Product modules and screens

Area	Required capabilities	Command-center treatment
Authentication	Register, login, password validation, reset	Role-aware entry
Admin/staff	Role controls; admin creates staff	Permission-aware staff directory
Customers	Create/manage; customer isolation	Searchable workspace
Products	Type, sale/cost price, notes	Product catalog
Variants	Variant options and price additions	Variant editor and price preview
Plans	Interval, minimum quantity, availability and lifecycle settings	Plan catalog/status
Quotations	Templates, validity, lines and customer offer	Draft/expiring/accepted queue
Subscriptions	Number, dates, payment terms, lines and state transitions	List, detail and timeline
Billing	Identify due subscriptions and generate invoices	Billing Center with batch results
Invoices	Draft, confirm, cancel, send, print, totals and status	Register and overdue queue
Taxes	Admin-configured rules	Tax settings and invoice breakdown
Discounts	Fixed/percentage, thresholds, dates, usage cap and scope	Rules and usage
Payments	Method, amount, date, note and invoice association	Ledger and reconciliation
Reports	Active subscriptions, revenue, payments, outstanding, overdue	Finance dashboard and filters
Audit	Trace important changes	Recent activity timeline

Navigation

Overview; Customers; Products & Variants; Plans; Quotations; Subscriptions; Billing Center; Invoices; Payments; Reports; Discounts; Tax Rules; Staff/Settings. The customer portal has reduced navigation for the authenticated customer's own quotations, subscriptions and invoices.

UX direction

Premium enterprise SaaS: restrained navy/indigo, light neutral surfaces, clear typography, accessible contrast, consistent spacing, useful tables and filters, responsive layouts, loading/empty/error states, and clear success feedback. Avoid excessive gradients or motion. The overview surfaces active subscriptions, recurring revenue where data supports it, collections, outstanding and overdue totals, revenue trend, due billing queue and recent activity.

4. Functional and non-functional requirements traceability

This matrix checks the proposed design against every requirement stated in the supplied problem statement. "MVP" means implement the core behavior in six hours where possible; "verify" means test it before claiming it works. This is design validation, not evidence that software has already been implemented.

ID	Source requirement	Proposed implementation	Six-hour treatment / validation
F01	Customer registration	Supabase Auth sign-up plus profile/customer creation	MVP; test success and duplicate email
F02	Email unique per account	Auth unique email and database constraint	MVP; test duplicate registration
F03	Password >8 chars with uppercase, lowercase, special character	Client and server validation; Auth policy	MVP; test invalid and valid cases
F04	Password reset link to registered email	Supabase reset flow with configured email delivery and redirect	Configure/test; no delivery claim without provider verification
F05	Admin, staff, customer roles	Profile role plus server-side authorization	MVP; test each role
F06	Only admin adds staff	Protected staff-creation operation	MVP; test staff/customer denial
F07	Customer sees only own records	Ownership checks and PostgreSQL RLS	MVP; cross-customer negative test
F08	Product name/type/sale price/cost price/notes	Products table and admin forms	MVP
F09	Product variants and price additions	Product variants with explicit price adjustment	MVP; test price calculation
F10	Daily/weekly/monthly/yearly plans	Enumerated interval and shared date utility	MVP; test each interval
F11	Minimum quantity and availability dates	Plan fields and validation	MVP
F12	Plan-specific auto-close, early close, pause and renew behavior	Plan policy fields and guarded lifecycle service	MVP core transitions; edge cases tested
F13	Quotation templates with plan/product lines	Template and template-line records	MVP simplified templates
F14	Quotation validity period	`valid_until` and acceptance validation	MVP; reject expired quotation
F15	Subscription number/customer/plan/start/expiry/payment terms	Subscription record with unique number and required fields	MVP
F16	Subscription lines: product, quantity, unit price, taxes, amount	Subscription items and tax snapshots	MVP
F17	Draft → Quotation → Confirmed → Active → Closed	Explicit state machine with guarded transitions	MVP; transition tests
F18	Team can see subscription status	Permission-filtered list/details	MVP
F19	Invoice generated from subscription	Billing service creates invoice and item snapshots	MVP
F20	Invoice includes customer, lines, taxes and totals	Invoice and invoice-item snapshots	MVP
F21	Invoice Draft → Confirmed → Paid	Invoice state plus payment balance logic	MVP; partial-payment extension modeled explicitly
F22	Confirm, cancel, send and print invoices	Protected actions, email integration, print/PDF view	Confirm/cancel/print MVP; send requires verified provider
F23	No duplicate or skipped billing	Due-date selection, unique subscription-period key, transaction and retry-safe processing	Critical MVP; repeated-batch test
F24	Admin-only fixed/percentage discounts	Protected discount management	MVP
F25	Discount minimum purchase or quantity	Eligibility evaluator	MVP
F26	Discount start/end dates and usage limit	Date/usage checks with atomic usage recording	MVP; test concurrent use if possible
F27	Discount applies to specific products or whole subscriptions	Scope/eligibility relation	MVP for defined scopes
F28	Discount automatically appears on next invoice	Billing engine evaluates active eligible rules	MVP; invoice breakdown test
F29	Discount stops when expired/exhausted	Eligibility evaluator checks dates and remaining uses	MVP
F30	Payments record method/amount/date/note against invoice	Payment ledger with invoice foreign key	MVP
F31	Show paid and outstanding invoices	Invoice balance/status derived from payments	MVP
F32	Overdue invoices visible for follow-up	Due date plus unpaid balance query and filters	MVP
F33	Finance view: active subscriptions	Aggregated dashboard query	MVP
F34	Finance view: revenue and payments	Defined collected-revenue metric and payment aggregates	MVP
F35	Finance view: outstanding and overdue	Outstanding balance and overdue aggregates	MVP
F36	Filters and summary view	Server-side filters, pagination and dashboard cards	MVP basic filters
N01	Pages under two seconds with thousands of subscriptions	Indexes, pagination, narrow selects, DB aggregation and measured query performance	Design supports; benchmark required
N02	Reliable billing/no duplicate or skipped invoices	Unique constraints, transactions, idempotent batch and error reporting	Critical; test failure/retry
N03	Role/data security	Server-side authorization, RLS, no service key in browser	Critical; negative permission tests
N04	Correct financial calculations	Integer minor units, deterministic tax/discount service, stored snapshots	Critical; unit tests and rounding policy

ID	Source requirement	Proposed implementation	Six-hour treatment / validation
N05	Usable unified system	Shared UI shell, coherent modules, status feedback and responsive layouts	MVP; human walkthrough
N06	Scalable/maintainable enough for stated scale	Modular monolith, relational schema, indexes and service boundaries	Appropriate architecture; production scale needs testing

Validation conclusion

The merged concept preserves every functional requirement and provides a design path for each non-functional requirement. No requirement is intentionally dropped. Actual compliance remains conditional on implementation and passing tests. Email delivery, scheduled production billing and sub-two-second performance at thousands of records depend on configuration, deployment and measurement; mark them unverified until tested.

5. Technology stack and rationale

Layer	Selected technology	Reason
Full-stack framework	Next.js App Router	One application for UI and server endpoints
Language	TypeScript	Shared contracts and safer parallel development
UI	Tailwind CSS + shadcn/ui + Lucide	Fast, consistent polished components
Forms/validation	React Hook Form + Zod	Reusable forms and server-side validation
Database	Supabase PostgreSQL	Relational financial data, constraints, transactions, indexes
Authentication	Supabase Auth	Email/password and reset flow; provider configuration required
Authorization	Server-side role checks + PostgreSQL RLS	Least privilege and customer isolation
Charts	Recharts	Dashboard visualizations
Dates	date-fns plus explicit billing rules	Shared date calculations
Tests	Vitest and integration checks	Fast unit tests for financial/lifecycle logic
Hosting	Vercel	Straightforward Next.js deployment
Collaboration	GitHub branches and pull requests	Shared source of truth and controlled integration

Use a **modular monolith**, not microservices. Keep Route Handlers thin; put business rules in feature services; validate every request; use a server-only database client for privileged operations. Never expose a Supabase service-role key to browser code.

Date and billing rules

A single billing-period utility handles daily, weekly, monthly and yearly cycles. Define month-end behavior (e.g. a subscription starting on the 31st) and leap-year behavior explicitly. Do not rely on accidental JavaScript date overflow.

Financial rules

Represent money in integer minor units (paise for INR) or a carefully tested decimal type. Never use binary floating point for financial totals. Define whether tax is inclusive or exclusive and define rounding at line or invoice level before implementation. The source does not prescribe tax rates; admins configure them.

Deployment and secrets

Use `.env.example` with variable names only. Keep secrets in local environment files and deployment environment settings. Verify Supabase URL/keys and email redirect URLs. A local demo fallback is not equivalent to production email delivery.

6. System architecture

The system is a modular monolith with three user experiences and one shared backend/data layer:

• **Presentation:** Admin console, staff workspace and customer portal.

• **Application/API:** Next.js App Router, Route Handlers, Zod validation, authentication and authorization.

• **Business modules:** customers, products, variants, plans, quotations, subscriptions and lifecycle transitions.

• **Financial modules:** billing, invoices, taxes, discounts, payments and reconciliation.

• **Intelligence:** finance dashboard, reports, activity/audit events; optional read-only AI summaries.

• **Data:** Supabase PostgreSQL, Supabase Auth, RLS, constraints, indexes and transactions.

All UI paths call the same service/API contracts. The browser never receives privileged database credentials. Financial operations are server-side and transactional.

7. Database design

Recommended tables

• `profiles`: auth user ID, email, display name, role, timestamps.

• `customers`: customer account/contact/company details and optional linked auth user.

• `products`: name, type, sale price, cost price, notes, active state.

• `product_variants`: product ID, variant name/value, price adjustment.

• `plans`: name, billing interval, minimum quantity, availability dates, lifecycle policy flags.

• `quotation_templates`: template name, default validity and metadata.

• `quotation_template_items`: product/plan references, quantity and price defaults.

• `quotations`: unique number, customer, template, status, valid-until, totals.

• `quotation_items`: product/plan, quantity, unit price and tax snapshot.

• `subscriptions`: unique number, customer, plan, status, start/expiry/next billing dates, payment terms, renewal and lifecycle settings.

• `subscription_items`: product/variant, quantity, unit price and tax snapshot.

• `tax_rules`: name, rate, tax behavior, active dates/status.

• `discounts`: type, value, threshold, scope, dates, usage cap and active state.

• `discount_usages`: discount, invoice, customer, amount and use timestamp.

• `invoices`: unique number, subscription, customer snapshot, billing period, due date, subtotal, discount, tax, total, paid and balance/status.

• `invoice_items`: product description snapshot, quantity, unit price, discount/tax breakdown and line total.

• `payments`: invoice, amount, method, date, note, reference and recorded-by.

• `audit_logs`: actor, action, entity type/ID, timestamp and safe metadata.

• `billing_runs`: run ID, timestamps, counts, status and error summary.

Integrity constraints and indexes

• Unique normalized email; unique subscription number and invoice number.

• Foreign keys for all parent/child records.

• Positive quantity and non-negative amount constraints where appropriate.

• Valid date-range checks.

• Index `subscriptions(status, next_billing_date)`.

• Index `invoices(status, due_date)` and `invoices(customer_id, created_at)`.

• Index `payments(invoice_id, payment_date)`.

• Index foreign-key columns and common filter/sort columns.

• Critical idempotency key: unique `(subscription_id, billing_period_start, billing_period_end)` for generated subscription invoices. If multiple invoice types exist, include invoice type/source in the key.

Data snapshots

Store invoice/customer/product descriptions, unit prices, discounts and tax details as snapshots on invoice lines. Later product price or customer address changes must not rewrite an issued invoice.

Payment balance

Outstanding = invoice total – valid recorded payments, bounded at zero only if overpayment policy says so. For the MVP, prevent payments greater than remaining balance unless explicit credit/overpayment handling exists. Derive `paid`, `partially_paid` and `outstanding` from balance while preserving the required Draft/Confirmed/Paid lifecycle.

Duplicate billing protection

The billing service selects due active subscriptions, determines an exact period, and attempts invoice creation inside a transaction. The unique period constraint is the final guard. Repeated requests return the existing invoice or an already-processed result, not a second invoice. Advance `next_billing_date` only after successful invoice creation in the same transaction. Record per-subscription failures so one bad record does not silently stop the batch.

8. Core workflows and state transitions

Main vertical slice

• Admin configures product, variant, plan, tax rule and discount.

• Customer registers; profile is linked to customer record.

• Staff creates quotation from a template and adds product/plan lines.

• Customer accepts while quotation is valid.

• System converts accepted quotation into a subscription, preserving agreed line prices and terms.

• Subscription moves Draft → Quotation → Confirmed → Active.

• Billing Center finds due active subscriptions and processes a billing run.

• Invoice lines, tax, discount and total are calculated by deterministic server-side code.

• Invoice is confirmed and made available to customer; sending requires configured email delivery.

• Staff records payment against the correct invoice.

• Invoice balance/status, customer portal, finance metrics and audit timeline update.

• Finance lead filters paid, outstanding and overdue invoices and reviews active subscription/revenue summaries.

Subscription state machine

`Draft` → `Quotation` → `Confirmed` → `Active` → `Closed`, with permitted side transitions to `Paused` and `Expired`, and renewal returning an eligible subscription to an active period. Each transition is validated by a service. A closed or paused subscription is not billed. Plan settings determine whether a subscription closes at plan end or can be renewed/paused/closed early.

Invoice state machine

Required states: `Draft` → `Confirmed` → `Paid`; cancellation is an allowed action before settlement subject to business rules. Partial payments are represented through a computed partial balance/status without falsely marking the invoice paid. Paid invoices must not be silently edited or cancelled.

Discounts

At invoice generation, evaluate active dates, eligible product/subscription scope, minimum purchase or quantity, usage limit and whether the customer/invoice already consumed the rule. Apply usage atomically with invoice creation to avoid concurrent overuse. Keep calculation deterministic and record the applied discount breakdown.

Taxes

Use admin-defined tax rules and a clearly specified inclusive/exclusive method. Preserve rate and amount snapshots on invoice lines. The source does not define legal tax rules, so the implementation must not imply compliance with an unspecified jurisdiction.

9. Four-member shared-repository plan

All four developers work in **one GitHub repository**, using separate feature branches and separate local working copies (or Git worktrees). Do not have four people/agents editing one shared working directory simultaneously. They share the repository and integrate continuously.

Workstream ownership

Member	Primary ownership	Concrete deliverables
M1 — Technical lead/integration	Bootstrap, app shell, shared contracts, dashboard integration, integration tests and deployment	Next.js setup, contracts, navigation, dashboard, main branch health
M2 — Frontend/UX	Design system, customer/product/plan/quotation screens, customer portal, tables/forms	Responsive pages, reusable components, loading/empty/error states
M3 — Backend/database	Supabase, migrations, auth, roles/RLS, customer/product/plan APIs, seed data	Schema, policies, validation, API foundations, permission tests
M4 — Subscription/billing	Subscription lifecycle, billing periods, invoice generation, taxes, discounts, payments	Billing service, state machine, idempotency, financial tests

Ownership boundaries

• M1 owns `app/layout.tsx`, global shell, navigation, shared API/type contracts and integration.

• M2 owns `components/ui/`, shared form/table primitives and assigned page UI; coordinate before changing global CSS.

• M3 owns `supabase/migrations/`, `lib/supabase/`, auth/permission modules and core database/API implementation.

• M4 owns `features/subscriptions/`, `features/billing/`, `features/invoices/` and `features/payments/` business services.

• `package.json`, global CSS, shared types and migrations are protected files: only their named owner changes them after agreement.

Suggested repository

``text subscript-one/ app/ (auth)/login/ register/ (dashboard)/dashboard/ customers/ products/ plans/ quotations/ subscriptions/ billing/ invoices/ payments/ reports/ portal/subscriptions/ invoices/ api/ components/ ui/ layout/ tables/ forms/ dashboard/ features/ customers/ products/ plans/ quotations/ subscriptions/ billing/ invoices/ payments/ analytics/ lib/ supabase/ auth/ permissions/ validation/ utils/ types/ database.ts api.ts common.ts billing.ts supabase/ migrations/ seed.sql tests/ docs/ ARCHITECTURE.md API_CONTRACTS.md DATABASE.md DEVELOPMENT.md tests/ billing/ subscriptions/ permissions/ .env.example README.md package.json ``

API contracts agreed before parallel coding

• `GET/POST /api/customers`

• `GET/POST /api/products`

• `GET/POST /api/plans`

• `GET/POST /api/quotations`

• `POST /api/quotations/:id/convert`

• `GET/POST /api/subscriptions`

• `PATCH /api/subscriptions/:id` for guarded transitions

• `POST /api/billing/process`

• `GET/POST /api/invoices`

• `POST /api/invoices/:id/confirm`

• `POST /api/invoices/:id/cancel`

• `POST /api/payments`

• `GET /api/dashboard`

• `GET /api/reports/revenue`

Define DTOs once in `types/` and publish sample request/response shapes in `docs/API_CONTRACTS.md`. The frontend may begin with mock services satisfying the same TypeScript interfaces, then replace them with real API calls without changing page contracts.

Git workflow

• `main` remains runnable.

• Branch names: `feature/member2-products-ui`, `feature/member3-auth-db`, `feature/member4-billing`.

• Commit messages: `feat:`, `fix:`, `ui:`, `test:`, `docs:`.

• First integration within hour one; integrate at least every 30–45 minutes.

• Small commits and short PRs; M1 reviews integration.

• Pull/rebase from main before pushing; communicate before shared-file changes.

• Keep a last-known-good commit. If a merge breaks the app, revert/isolate the offending commit and restore main before continuing.

10. Six-hour execution schedule

Time	M1 — Lead	M2 — UI	M3 — Backend/DB	M4 — Billing
0:00–1:00	Bootstrap app, shell, types/contracts	Design tokens, sidebar, base pages	Supabase, auth, initial schema/RLS	Billing types, date rules, calculation tests
1:00–2:00	Dashboard skeleton and integration	Customer/product/plan UI	Customer/product/plan APIs	Subscription model and quotation conversion
2:00–3:00	Dashboard API wiring/activity	Quotation/subscription UI	Invoice/payment API foundations	Invoice generator, tax/discount logic
3:00–4:00	Integrate vertical slice	Invoice/payment/portal UI	Payment persistence, permissions	Idempotent billing and reconciliation
4:00–5:00	Integration and analytics	Polish, filters, responsive states	RLS/constraint tests, seed data	Renewal/pause/overdue edge cases
5:00–6:00	Final integration, deploy, demo lead	UI defects and portal check	Security/data integrity QA	Financial correctness and billing QA

Checkpoints

• **Hour 1:** everyone runs the same repository; database connection and shared contracts exist.

• **Hour 2:** product/customer/plan basics and subscription service are connected.

• **Hour 3:** quotation conversion and invoice generation work.

• **Hour 4:** payment recording updates invoice and dashboard.

• **Hour 5:** critical tests pass; feature freeze.

• **Hour 6:** bug fixes, rehearsal and deployment/demo fallback.

Do not postpone integration to the last hour. Do not let all four members spend the first hour debating tools. M1 establishes the minimal skeleton and contracts; other members build within agreed boundaries.

11. AI coding-agent operating rules

Each member may use an IDE coding agent, but the agent is scoped to that member's workstream and branch. Agents must read README and architecture/API/database contracts before editing. Agents must not independently redesign the stack, alter shared types, change migrations owned by M3, or edit another member's files without coordination.

Agent task format: context → exact task → allowed files → forbidden files → existing interfaces → acceptance criteria → tests → report of changed files and dependencies.

Use agents for implementation, repetitive UI, tests, SQL drafts and code review. Human owners review authentication, RLS, SQL migrations, financial calculations and billing state transitions. Compile/test after each integration. Avoid agents editing a shared working directory concurrently; separate branches/worktrees are safer.

12. Testing and acceptance criteria

Mandatory test scenarios

• Duplicate email registration is rejected.

• Passwords failing any stated rule are rejected.

• Staff cannot add staff; only admin can.

• Customer A cannot access Customer B's subscription or invoice by changing an ID.

• Product variant price adjustment is included exactly once.

• Daily, weekly, monthly and yearly billing periods are correct, including month-end/leap-year policy.

• Expired quotation cannot be accepted.

• Invalid subscription transitions are rejected.

• Paused/closed/expired subscriptions are not billed.

• Re-running a billing batch does not create duplicate invoices.

• Failed invoice operation does not silently advance next billing date.

• Discount is rejected when out of date, below threshold or exhausted.

• Discount usage cannot exceed its cap under concurrent requests.

• Tax and discount totals match deterministic test fixtures.

• Partial payment reduces outstanding; full payment marks paid.

• Overdue query excludes paid/cancelled invoices.

• Dashboard totals reconcile with underlying invoice/payment records.

• Invoice print output contains customer, lines, taxes and totals.

• Password reset and invoice email are tested with configured email provider.

• Pagination and dashboard query timings are measured with a larger seed dataset.

Example deterministic calculation fixture

Example only: two units at ■10,000 plus a ■560 variant adjustment per unit produce ■21,120 subtotal. A 10% discount gives ■2,112. With an illustrative 18% exclusive tax on ■19,008, tax is ■3,421.44 and total is ■22,429.44. Confirm currency precision and rounding policy before using this fixture; the problem statement itself does not prescribe tax rates.

Performance acceptance

The source sets a target of under two seconds per page with thousands of subscriptions. Use server-side pagination, appropriate indexes, selected columns and database aggregation. Seed thousands of representative rows if time allows and measure. A design intention is not a performance result; report measured response times only.

13. Risks, limits and mitigations

Risk	Mitigation
Scope exceeds six hours	Finish one complete vertical slice first; classify optional features

Risk	Mitigation
Merge conflicts	One owner per shared file; frequent small PRs
Inconsistent agent output	Shared docs/contracts and strict task scope
Duplicate billing	Unique period constraint + transaction + retry-safe batch
Incorrect tax/discount	Deterministic service, explicit tax policy and unit tests
Customer data leakage	RLS and server authorization; negative tests
Email provider not ready	Configure early; disclose fallback; do not claim email sent
Scheduled job not configured	Use real manual-triggered batch for demo; schedule later
Deployment outage	Local seeded demo fallback and stable commit
Slow queries	Index, paginate, aggregate and benchmark
Unsupported tax/legal assumptions	Make tax rules configurable; obtain jurisdiction requirements before production
Partial payment ambiguity	Track payment ledger and outstanding balance separately from final paid status

Explicit non-claims

Unless implemented and verified, do not claim production-grade scheduled billing; real payment gateway collection; actual email delivery; legal/tax compliance for a jurisdiction; guaranteed sub-two-second performance at production scale; or full enterprise scalability. The MVP can demonstrate core behavior with real persistence while honestly documenting these production tasks.

14. Three-minute demonstration

Prepare demo accounts: admin, staff/sales, finance staff and customer. Prepare a product with a price-adjusting variant, monthly plan, quotation template, one customer, one active subscription and one overdue invoice. Use synthetic records.

Time	Demo action	What judges should see
0:00–0:25	State manual-process problem and introduce Subscript One	One platform for full lifecycle
0:25–0:55	Admin creates/opens product, variant and plan	Configuration and variant price effect
0:55–1:25	Staff opens quotation and converts accepted offer	Quotation → subscription
1:25–2:05	Run Process Due Invoices	Real invoice with tax/discount breakdown and duplicate protection
2:05–2:35	Record payment	Invoice balance/status changes
2:35–3:00	Return to finance dashboard	Active subscriptions, collected, outstanding and overdue metrics reconcile

The strongest proof is one continuous persisted workflow: **Quotation** → **Subscription** → **Invoice** → **Payment** → **Dashboard update.** Avoid spending the demo on an AI feature or static charts. If email sending or scheduled automation is not configured, say so accurately.

15. Final implementation checklist

Foundation

- [ ] Create one shared GitHub repository and four feature branches.
- [ ] Initialize Next.js + TypeScript and confirm all four developers can run it.
- [ ] Create Supabase project and secure environment configuration.
- [ ] Agree shared types, API contracts and file ownership.
- [ ] Add initial migration, seed data and application shell.

Required business functionality

- [ ] Authentication, password policy and reset flow.
- [ ] Admin/staff/customer authorization and customer isolation.
- [ ] Admin-only staff creation.
- [ ] Products, variants and price adjustments.
- [ ] Plans with four billing intervals, minimum quantity and dates.
- [ ] Quotation templates, validity and conversion.
- [ ] Subscription number, dates, terms, lines and state transitions.
- [ ] Billing batch, idempotency and due-date updates.
- [ ] Invoice lifecycle, line/tax/discount snapshots, print and send integration.
- [ ] Admin discount rules and automatic eligibility evaluation.

• [ ] Payment ledger and outstanding/overdue calculations.

• [ ] Finance dashboard and filters.

Quality and demonstration

• [ ] Test security boundaries and customer isolation.

• [ ] Test financial calculations and billing-period edge cases.

• [ ] Test duplicate billing and retry behavior.

• [ ] Measure dashboard/page response time with larger data.

• [ ] Verify email delivery or disclose demo limitation.

• [ ] Seed realistic demo data and rehearse the three-minute flow.

• [ ] Freeze features before final hour and keep a known-good commit.

16. Final verdict

The merged Idea 1 + Idea 5 is the recommended direction. Idea 1 ensures the application remains a complete subscription management system and covers the actual requirements. Idea 5 improves the experience with a command center that makes those capabilities visible, actionable and compelling to demonstrate.

The correct balance is:

• Do not reduce the system to a dashboard.

• Do not build only CRUD screens.

• Do not substitute AI for deterministic financial logic.

• Do not omit any requirement from the design.

• Implement a working, testable core vertical slice within six hours and label production-dependent capabilities honestly.

Requirement validation result: The proposed design maps every functional and non-functional requirement in the supplied problem statement to a module, rule or validation test. Actual compliance remains conditional on implementation and passing the listed tests. The under-two-second performance target, email delivery and production scheduling cannot be certified by a design document alone.

Implementation gate: This document is a blueprint only. Do not begin application coding until the team approves product scope, tax/payment assumptions, stack, shared interfaces and ownership plan.